INDEX

Sr.No.	Title	Sign
	Module 1	
1	Basic Logic Gates	
2	Demorgan’s Theorem	
3	3 Variable K Map	
4	Half Adder	
5	Half Subtractor	
	Module 2	
6	JK Flipflop	
7	4:1 Multiplexer	
8	1:4 Demultiplexer	
9	4 bit counter	
10	4 bit shift register	

FYBSc IT/CS – CSD/DSA Practical File

MODULE 1

Practical 1: Basic Logic Gates

1. Aim

To implement and verify basic logic gates using Logisim Evolution.

2. Requirements

Logisim Evolution, Input Pins, Output Pins and basic logic gates.

3. Theory

Logic gates are electronic circuits that perform logical operations on binary inputs and produce a binary output.

4. Circuit Diagram

Implement **AND, OR, NOT, NAND, NOR, XOR and XNOR** gates in Logisim Evolution.

5. Truth Table

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

NOT Gate:

A	NOT A
0	1
1	0

6. Procedure

1. Open Logisim Evolution.
2. Place the required gates.
3. Connect Input Pins to the gates.
4. Connect the gate outputs to Output Pins.
5. Apply different input combinations and observe the outputs.

7. Result

Basic logic gates were successfully implemented and verified.

Practical 2: De Morgan's Theorem

1. Aim

To verify De Morgan's theorems using logic gates in Logisim Evolution.

2. Requirements

Logisim Evolution, AND, OR and NOT gates, Input Pins and Output Pins.

3. Theory

De Morgan's theorems are Boolean algebra rules used to simplify logical expressions.

4. Circuit Diagram

Implement both sides of each theorem and compare their outputs.

5. Truth Table

Theorem 1: (A + B)' = A'B'

A	B	(A+B)'	A'B'
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Theorem 2: (AB)' = A' + B'

A	B	(AB)'	A'+B'
0	0	1	1
0	1	1	1

A	B	(AB)'	A'+B'
1	0	1	1
1	1	0	0

6. Procedure

1. Construct the circuit for the left side.
2. Construct the circuit for the right side.
3. Apply all input combinations.
4. Compare both outputs.

7. Result

De Morgan's theorems were successfully verified.

Practical 3: 3-Variable K-Map

1. Aim

To simplify a 3-variable Boolean expression using K-map and verify it using Logisim Evolution.

2. Requirements

Logisim Evolution, Input Pins, Output Pin and required logic gates.

3. Theory

A Karnaugh Map is a graphical method used to simplify Boolean expressions by grouping adjacent 1s.

4. Given Expression

$$ F(A,B,C)=\Sigma m(0,1,2,3,6,7) $$

5. K-Map

AB \ C	**0**	**1**
00	1	1
01	1	1
11	1	1
10	0	0

6. Simplification

Groups:

- m0, m1, m2, m3 → **A'**
- m2, m3, m6, m7 → **B**

Therefore,

$$ \boxed{F=A'+B} $$

7. Truth Table

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

8. Procedure

1. Draw the 3-variable K-map.
2. Enter 1 at the given minterms.
3. Make the largest possible groups.
4. Obtain the simplified expression.
5. Implement the expression in Logisim Evolution.
6. Verify the output using the truth table.

9. Result

The given Boolean expression was successfully simplified and verified.

Practical 4: Half Adder

1. Aim

To implement and verify a Half Adder using the Half Adder component in Logisim Evolution.

2. Requirements

Logisim Evolution, Half Adder component, Input Pins and Output Pins.

3. Theory

A Half Adder adds two binary bits and produces **Sum** and **Carry** outputs.

4. Circuit Diagram

Use the **Half Adder component** with inputs A and B and outputs Sum and Carry.

5. Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

6. Procedure

1. Place the Half Adder component.
2. Connect inputs A and B.
3. Connect Sum and Carry to Output Pins.
4. Apply all input combinations.
5. Verify the outputs.

7. Result

The Half Adder was successfully implemented and verified.

Practical 5: Half Subtractor

1. Aim

To implement and verify a Half Subtractor using the Half Subtractor component in Logisim Evolution.

2. Requirements

Logisim Evolution, Half Subtractor component, Input Pins and Output Pins.

3. Theory

A Half Subtractor subtracts one binary bit from another and produces **Difference** and **Borrow** outputs.

4. Circuit Diagram

Use the **Half Subtractor component** with inputs A and B and outputs Difference and Borrow.

5. Truth Table

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

6. Procedure

1. Place the Half Subtractor component.
2. Connect inputs A and B.
3. Connect Difference and Borrow to Output Pins.
4. Apply all input combinations.
5. Verify the outputs.

7. Result

The Half Subtractor was successfully implemented and verified.

MODULE 2

Practical 6: JK Flip-Flop

1. Aim

To implement and verify a JK Flip-Flop using the JK Flip-Flop component in Logisim Evolution.

2. Requirements

Logisim Evolution, JK Flip-Flop component, Clock, Input Pins and Output Pins.

3. Theory

A JK Flip-Flop is a sequential circuit having two inputs J and K and is used to store one bit of information.

4. Circuit Diagram

Use the **JK Flip-Flop component** with J, K and Clock inputs and Q output.

5. Truth Table

J	K	Q(next)	Operation
0	0	Q	No change
0	1	0	Reset
1	0	1	Set
1	1	$\overline{Q}$	Toggle

6. Procedure

1. Place the JK Flip-Flop component.
2. Connect J, K and Clock.
3. Connect Q to an Output Pin.
4. Apply different combinations of J and K.

5. Apply clock pulses and observe Q.

7. Result

The JK Flip-Flop was successfully implemented and verified.

Practical 7: 4:1 Multiplexer

1. Aim

To implement and verify a 4:1 Multiplexer using the Multiplexer component in Logisim Evolution.

2. Requirements

Logisim Evolution, Multiplexer component, four inputs, two select lines and Output Pin.

3. Theory

A Multiplexer selects one of several input signals and sends the selected signal to a single output.

4. Circuit Diagram

Use the **Multiplexer component** with 4 inputs and 2 select lines.

5. Truth Table

S1	S0	Output
0	0	I0
0	1	I1
1	0	I2
1	1	I3

6. Procedure

1. Place the Multiplexer component.
2. Set the number of inputs to 4.
3. Connect I0–I3.
4. Connect select lines S1 and S0.
5. Observe the output for each select combination.

7. Result

The 4:1 Multiplexer was successfully implemented and verified.

Practical 8: 1:4 Demultiplexer

1. Aim

To implement and verify a 1:4 Demultiplexer using the Demultiplexer component in Logisim Evolution.

2. Requirements

Logisim Evolution, Demultiplexer component, Data Input, Select Lines and Output Pins.

3. Theory

A Demultiplexer takes one input and sends it to one of several outputs according to the select lines.

4. Circuit Diagram

Use the **Demultiplexer component** with 1 input, 4 outputs and 2 select lines.

5. Truth Table

S1	S0	Y0	Y1	Y2	Y3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

6. Procedure

1. Place the Demultiplexer component.
2. Set the number of outputs to 4.
3. Connect data input D.
4. Connect select lines S1 and S0.
5. Observe the selected output.

7. Result

The 1:4 Demultiplexer was successfully implemented and verified.

Practical 9: 4-Bit Counter

1. Aim

To implement and verify a 4-bit Counter using the Counter component in Logisim Evolution.

2. Requirements

Logisim Evolution, Counter component, Clock and Output.

3. Theory

A counter is a sequential circuit that counts clock pulses and produces a binary output.

4. Circuit Diagram

Use the **Counter component** and set **Data Bits = 4**.

5. Observation Table

Clock Pulse	Output
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000

Clock Pulse	Output
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
16	0000

6. Procedure

1. Place the Counter component.
2. Set **Data Bits = 4**.
3. Connect the Clock.
4. Apply clock pulses.
5. Observe the output sequence.

7. Result

The 4-bit Counter was successfully implemented and verified.

Practical 10: 4-Bit Shift Register

1. Aim

To implement and verify a 4-bit Shift Register using the Shift Register component in Logisim Evolution.

2. Requirements

Logisim Evolution, Shift Register component, Clock, Input and Output.

3. Theory

A shift register is a sequential circuit used to store and shift binary data from one position to another.

4. Circuit Diagram

Use the **Shift Register component** and set the number of bits to **4**.

5. Observation Table

For serial input **1, 0, 1, 1**:

Clock	Serial Input	Register Output
0	–	0000
1	1	0001
2	0	0010
3	1	0101
4	1	1011

Note: The exact displayed sequence can depend on the shift direction/settings selected in Logisim.

6. Procedure

1. Place the Shift Register component.
2. Set the number of bits to 4.
3. Connect the Clock and serial input.
4. Apply the input data.
5. Apply clock pulses.
6. Observe the shifting of data.

7. Result

The 4-bit Shift Register was successfully implemented and verified.